

Jenkins Pipeline Task 1

Task 1: Pipeline Job for VProfile-1 (Git Clone, SonarQube Traditional Install, Slack Alerts)

Outcome

A Jenkins pipeline job named vprofile-sonar-slack runs three stages against the VProfile-1 repository: Git Clone downloads the source code, SonarQube Analysis submits the code to a manually installed SonarQube server for a quality scan, and Slack Notification posts the build result to a Slack channel.

Objective

To build a pipeline job that pulls source code from GitHub, checks code quality against a traditionally installed (non-Docker) SonarQube server, and notifies the team on Slack as soon as the build finishes, replacing manual code review checks and manual status updates.

Prerequisites

- Jenkins master installed and reachable, with the Pipeline, Git, SonarQube Scanner, and Slack Notification plugins installed.
- A Linux server (can be the Jenkins server itself or a dedicated host) with at least 2 vCPU and 4 GB RAM for the traditional SonarQube install, plus Java (JDK 17) installed.
- SSH/console access to that server with sudo privileges to run the manual SonarQube installation.
- A Slack workspace with permission to create an app / incoming webhook, and a Slack channel to post build alerts to.
- Public read access (or credentials) to github.com/betawins/VProfile-1.git.

Step-by-Step Implementation

Step 1: Install SonarQube Server (Traditional / Manual Install)

MySQL Database Server or MySQL RDS instance..

```
yum update
sudo wget https://dev.mysql.com/get/mysql84-community-release-el9-1.noarch.rpm
sudo dnf install mysql84-community-release-el9-1.noarch.rpm -y
sudo rpm --import https://repo.mysql.com/RPM-GPG-KEY-mysql-2025
sudo yum install mysql-community-server
sudo systemctl start mysqld.service
mysql --version
```

 **Screenshot Evidence:** capture the console view for this step and insert it here.

```
[root@ip-172-31-24-201 ec2-user]# sudo wget https://dev.mysql.com/get/mysql84-community-release-el9-1.noarch.rpm
--2026-08-13 12:18:43-- https://dev.mysql.com/get/mysql84-community-release-el9-1.noarch.rpm
Resolving dev.mysql.com (dev.mysql.com)... 104.71.182.174, 2600:1408:c400:188c::2e31, 2600:1408:c400:1881::2e31
Connecting to dev.mysql.com (dev.mysql.com)|104.71.182.174|:443... connected.
HTTP request sent, awaiting response... 302 Moved Temporarily
Location: https://repo.mysql.com/mysql84-community-release-el9-1.noarch.rpm [following]
--2026-08-13 12:18:43-- https://repo.mysql.com/mysql84-community-release-el9-1.noarch.rpm
Resolving repo.mysql.com (repo.mysql.com)... 23.33.203.94, 2600:1408:c400:168d::1d68, 2600:1408:c400:1692::1d68
Connecting to repo.mysql.com (repo.mysql.com)|23.33.203.94|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 13139 (13K) [application/x-redhat-package-manager]
Saving to: 'mysql84-community-release-el9-1.noarch.rpm'

mysql84-community-release-el9-1.noarch 100%[=====>] 12.83K --.-KB/s in 0s

2026-08-13 12:18:43 (262 MB/s) - 'mysql84-community-release-el9-1.noarch.rpm' saved [13139/13139]
```

```
[root@ip-172-31-24-201 ec2-user]# sudo dnf install mysql84-community-release-el9-1.noarch.rpm -y
Amazon Linux 2023 repository 64 MB/s | 75 MB
Amazon Linux 2023 Kernel Livepatch repository 471 kB/s | 69 kB
Dependencies resolved.

=====
Package Architecture Version Repository
=====
Installing:
mysql84-community-release noarch el9-1 @commandline
=====
Transaction Summary
=====
Install 1 Package

Total size: 13 k
Installed size: 14 k
Downloading Packages:
Running transaction check
Transaction check succeeded.
Running transaction test
Transaction test succeeded.
Running transaction
Preparing :
Installing : mysql84-community-release-el9-1.noarch
Verifying : mysql84-community-release-el9-1.noarch
```

```
[root@ip-172-31-24-201 ec2-user]# sudo rpm --import https://repo.mysql.com/RPM-GPG-KEY-mysql-2025
[root@ip-172-31-24-201 ec2-user]# sudo dnf clean all
sudo dnf makecache
17 files removed
Amazon Linux 2023 repository
Amazon Linux 2023 Kernel Livepatch repository
MySQL 8.4 LTS Community Server
MySQL Connectors Community
MySQL Tools 8.4 LTS Community
Metadata cache created.
```

```
[root@ip-172-31-24-201 ec2-user]# sudo dnf install mysql-community-server -y
Last metadata expiration check: 0:00:09 ago on Thu Aug 13 12:20:08 2026.
Dependencies resolved.

=====
Package Architecture Version Repository Size
=====
Installing:
mysql-community-server x86_64 8.4.11-1.el9 mysql-8.4-lts-community 52 M
Installing dependencies:
mysql-community-client x86_64 8.4.11-1.el9 mysql-8.4-lts-community 3.4 M
mysql-community-client-plugins x86_64 8.4.11-1.el9 mysql-8.4-lts-community 1.5 M
mysql-community-common x86_64 8.4.11-1.el9 mysql-8.4-lts-community 581 k
mysql-community-icu-data-files x86_64 8.4.11-1.el9 mysql-8.4-lts-community 2.3 M
mysql-community-libs x86_64 8.4.11-1.el9 mysql-8.4-lts-community 1.5 M
=====
Transaction Summary
=====
Install 6 Packages

Total download size: 62 M
Installed size: 340 M
Downloading Packages:
(1/6): mysql-community-common-8.4.11-1.el9.x86_64.rpm 37 MB/s | 581 kB 00:00
(2/6): mysql-community-client-plugins-8.4.11-1.el9.x86_64.rpm 57 MB/s | 1.5 MB 00:00
(3/6): mysql-community-client-8.4.11-1.el9.x86_64.rpm 71 MB/s | 3.4 MB 00:00
```

```
[root@ip-172-31-24-201 ec2-user]# sudo systemctl enable --now mysqld
[root@ip-172-31-24-201 ec2-user]# sudo systemctl status mysqld
● mysqld.service - MySQL Server
   Loaded: loaded (/usr/lib/systemd/system/mysqld.service; enabled; preset: disabled)
   Active: active (running) since Thu 2026-08-13 12:20:42 UTC; 5s ago
     Docs: man:mysqld(8)
           http://dev.mysql.com/doc/refman/en/using-systemd.html
   Process: 23127 ExecStartPre=/usr/bin/mysqld_pre_systemd (code=exited, status=0/SUCCESS)
  Main PID: 28072 (mysqld)
    Status: "Server is operational"
     Tasks: 35 (limit: 1059)
    Memory: 453.5M
       CPU: 3.874s
   CGroup: /system.slice/mysqld.service
           └─28072 /usr/sbin/mysqld

Aug 13 12:20:34 ip-172-31-24-201.ec2.internal systemd[1]: Starting mysqld.service - MySQL Server...
Aug 13 12:20:42 ip-172-31-24-201.ec2.internal systemd[1]: Started mysqld.service - MySQL Server.
[root@ip-172-31-24-201 ec2-user]#
```

Configure the MySQL Root Password

You will see default MySQL root password

```
grep 'temporary' /var/log/mysqld.log
```

Login to mysql using the default password

```
mysql -u root -p
```

Now replace the default password with a new and strong password

```
ALTER USER 'root'@'localhost' IDENTIFIED BY 'Admin@123';
```

```
flush privileges;
```

Test Using new password

```
mysql -u root -p
```

 **Screenshot Evidence:** capture the console view for this step and insert it here.

```
[root@ip-172-31-24-201 ec2-user]# grep 'temporary' /var/log/mysqld.log
2026-08-13T12:20:38.504963Z 6 [Note] [MY-010454] [Server] A temporary password is generated for root@localhost: R6RlYt_uYB!q
[root@ip-172-31-24-201 ec2-user]# mysql -u root -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 13
Server version: 8.4.11

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> ALTER USER 'root'@'localhost' IDENTIFIED BY 'Admin@123';
Query OK, 0 rows affected (0.01 sec)

mysql> flush privileges;
Query OK, 0 rows affected (0.01 sec)

mysql> exit
```

Create a new sonar database and Create a local and a remote user

```
CREATE DATABASE sonar CHARACTER SET utf8 COLLATE utf8_general_ci;
```

```
CREATE USER sonar@localhost IDENTIFIED BY 'Sonar@123';
```

```
CREATE USER sonar@'%' IDENTIFIED BY 'Sonar@123';  
show databases;  
SELECT User FROM mysql.user;  
FLUSH PRIVILEGES;  
QUIT
```

 **Screenshot Evidence:** capture the console view for this step and insert it here.

```
mysql> show databases;  
+-----+  
| Database |  
+-----+  
| information_schema |  
| mysql |  
| performance_schema |  
| sys |  
+-----+  
4 rows in set (0.01 sec)  
  
mysql> CREATE DATABASE sonar CHARACTER SET utf8 COLLATE utf8_general_ci;  
Query OK, 1 row affected, 2 warnings (0.00 sec)
```

```
mysql> CREATE USER sonar@localhost IDENTIFIED BY 'Sonar@123';  
Query OK, 0 rows affected (0.01 sec)  
  
mysql> CREATE USER sonar@'%' IDENTIFIED BY 'Sonar@123';  
Query OK, 0 rows affected (0.00 sec)  
  
mysql> show users;
```

```
mysql> GRANT ALL ON sonar.* TO sonar@localhost;  
Query OK, 0 rows affected (0.01 sec)  
  
mysql> GRANT ALL ON sonar.* TO sonar@'%;  
Query OK, 0 rows affected (0.00 sec)
```

```
mysql> show databases;
+-----+
| Database          |
+-----+
| information_schema |
| mysql              |
| performance_schema |
| sonar              |
| sys                |
+-----+
5 rows in set (0.01 sec)
```

```
mysql> SELECT User FROM mysql.user;
+-----+
| User              |
+-----+
| sonar             |
| mysql.infoschema  |
| mysql.session     |
| mysql.sys         |
| root              |
| sonar             |
+-----+
6 rows in set (0.00 sec)

mysql> FLUSH PRIVILEGES;
Query OK, 0 rows affected (0.01 sec)

mysql> exit
Bye
```

Download and unzip the SonarQube distribution ZIP directly (no Docker container) into /opt.

```
cd /opt
sudo wget https://binaries.sonarsource.com/Distribution/sonarqube/sonarqube-7.8.zip
sudo unzip sonarqube-sonarqube-7.8
sudo mv sonarqube-sonarqube-7.8 sonarqube
sudo chown -R ec2-user: ec2-user /opt/sonarqube

cd /opt/sonarqube/bin/linux-x86-64
./sonar.sh start
./sonar.sh status
```

 **Screenshot Evidence:** capture the console view for this step and insert it here.

```
cd /opt
sudo wget https://binaries.sonarsource.com/Distribution/sonarqube/sonarqube-7.8.zip
```

```
[root@ip-172-31-24-201 opt]# chown ec2-user:ec2-user sonarqube
[root@ip-172-31-24-201 opt]# ll
total 200916
drwxr-xr-x.  4 root      root              33 Aug  1 04:14 aws
drwxr-xr-x. 11 ec2-user  ec2-user          141 Jun 17  2019 sonarqube
-rw-r--r--.  1 root      root        205736714 Feb 16  2022 sonarqube-7.8.zip
[root@ip-172-31-24-201 opt]#
```

```
[root@ip-172-31-24-201 opt]# cd sonarqube/
[root@ip-172-31-24-201 sonarqube]# ll
total 24
-rw-r--r--. 1 root root  7651 Jun 17  2019 COPYING
drwxr-xr-x. 6 root root   94 Jun 17  2019 bin
drwxr-xr-x. 2 root root   50 Jun 17  2019 conf
drwxr-xr-x. 2 root root   24 Jun 17  2019 data
drwxr-xr-x. 7 root root  131 Jun 17  2019 elasticsearch
drwxr-xr-x. 4 root root   40 Jun 17  2019 extensions
drwxr-xr-x. 6 root root  123 Jun 17  2019 lib
drwxr-xr-x. 2 root root   24 Jun 17  2019 logs
drwxr-xr-x. 2 root root   24 Jun 17  2019 temp
drwxr-xr-x. 7 root root 16384 Jun 17  2019 web
```

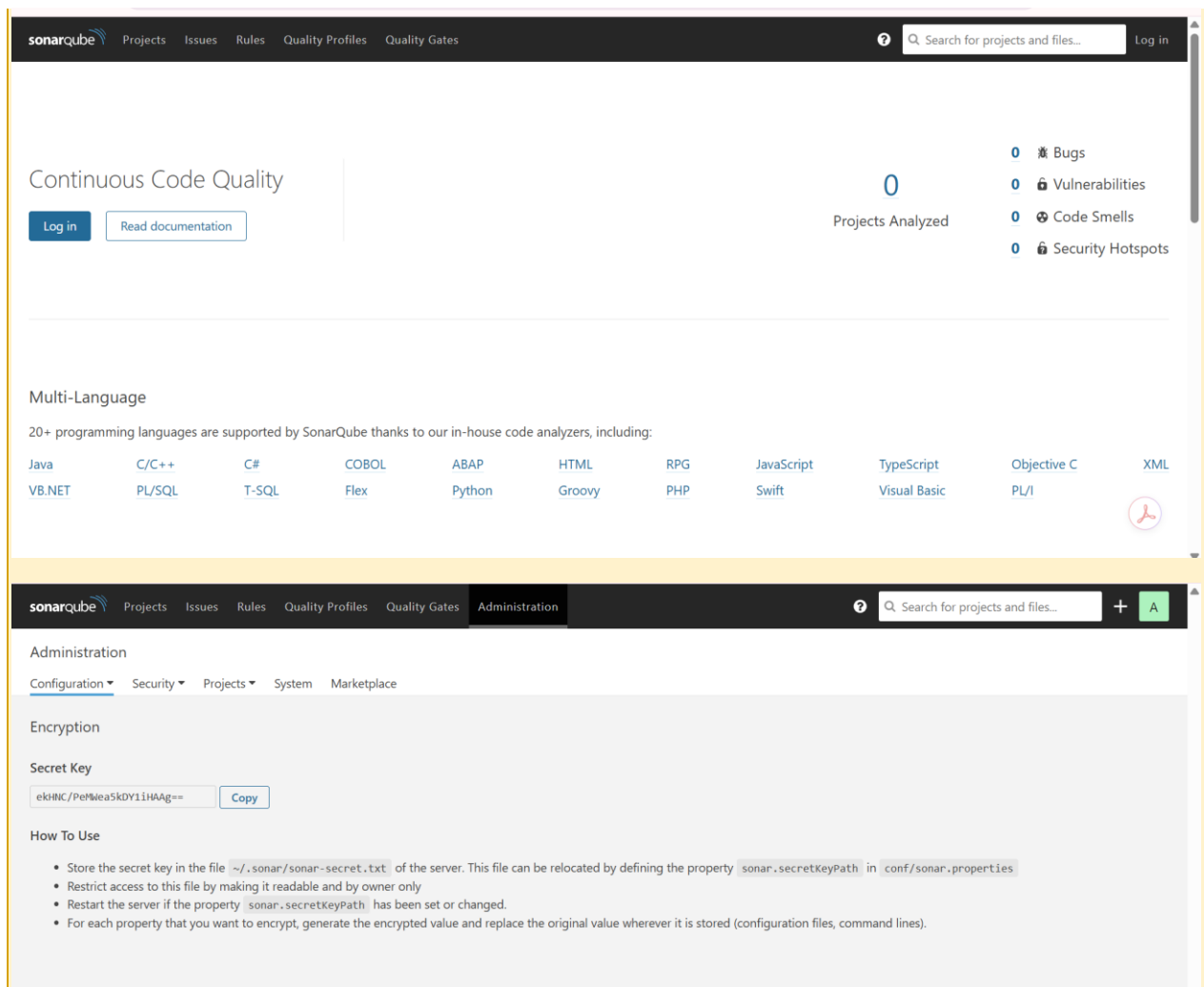
```
[root@ip-172-31-24-201 sonarqube]# cd bin/
[root@ip-172-31-24-201 bin]# ll
total 0
drwxr-xr-x. 2 root root  25 Jun 17  2019 jsw-license
drwxr-xr-x. 3 root root  48 Jun 17  2019 linux-x86-64
drwxr-xr-x. 3 root root  48 Jun 17  2019 macosx-universal-64
drwxr-xr-x. 3 root root 167 Jun 17  2019 windows-x86-64
```

```
[root@ip-172-31-24-201 linux-x86-64]# ll
total 132
drwxr-xr-x. 2 root root   27 Jun 17  2019 lib
-rwxr-xr-x. 1 root root 16393 Jun 17  2019 sonar.sh
-rwxr-xr-x. 1 root root 111027 Jun 17  2019 wrapper
[root@ip-172-31-24-201 linux-x86-64]# ./sonar.sh
Usage: ./sonar.sh { console | start | stop | force-stop | restart | status | dump }
[root@ip-172-31-24-201 linux-x86-64]# ./sonar.sh start
Starting SonarQube...
Started SonarQube.
```

Step 2: Verify SonarQube Is Reachable and Create a Token

Parameter	Value
URL	http://<sonarqube-server-ip>:9000
Default login	admin / admin (change on first login)
Token location	My Account -> Security -> Generate Tokens
Token name	jenkins-integration-token

 **Screenshot Evidence:** capture the console view for this step and insert it here.



Step 3: Configure the SonarQube Server Inside Jenkins

Parameter	Value
Location	Manage Jenkins -> System -> SonarQube servers
Name	sonar-server
Server URL	http://<sonarqube-server-ip>:9000
Server authentication token	Credential built from the token generated in Step 2

Also register the scanner tool under Manage Jenkins -> Tools -> SonarQube Scanner installations, name it sonar-scanner, and let Jenkins install it automatically.

 **Screenshot Evidence:** capture the console view for this step and insert it here.

 **Jenkins**

Manage Jenkins

Plugins

Search

Settings

User

Updates

Available plugins

Installed plugins

Advanced settings

Download progress

Download progress

Download progress

Preparation

- Checking internet connectivity
- Checking update center connectivity
- Success

SonarQube Scanner

Success

Loading plugin extensions

Success

[Go back to the top page](#)
(you can start using the installed plugins right away)

☐ Restart Jenkins when installation is complete and no jobs are running

< Add Secret text

X

Scope ?

Global (Jenkins, nodes, items, all child items, etc)

Secret

.....

ID ?

sonarqube

Description ?

sonarqube

Create

X

Name

sonar-server

Server URL

Default is http://localhost:9000

http://13.220.59.156:9000

Server authentication token

SonarQube authentication token. Mandatory when anonymous access is disabled.

sonarqube

+ Add

Advanced

Login to Jenkins server and install sonarqube scanner.


```
drwxr-xr-x. 4 root root      33 Aug  1 04:14 aws
drwxr-xr-x. 6 root root      51 May  7 2021 sonar-scanner-4.6.2.2472-linux
-rw-r--r--. 1 root root 43099390 Feb 16 2022 sonar-scanner-cli-4.6.2.2472-linux.zip
[ec2-user@ip-172-31-40-113 opt]$ sudo mv sonar-scanner-4.6.2.2472-linux /opt/sonar_scanner
[ec2-user@ip-172-31-40-113 opt]$ ll
total 42092
drwxr-xr-x. 4 root root      33 Aug  1 04:14 aws
-rw-r--r--. 1 root root 43099390 Feb 16 2022 sonar-scanner-cli-4.6.2.2472-linux.zip
drwxr-xr-x. 6 root root      51 May  7 2021 sonar_scanner
[ec2-user@ip-172-31-40-113 opt]$
```

i-0e340af6d7d1ae89b (Jenkins-Master[])

PublicIPs: 100.31.57.152 PrivateIPs: 172.31.40.113

SonarQube Scanner installations

+ Add SonarQube Scanner

⋮ SonarQube Scanner

Name

sonar_scanner

SONAR_RUNNER_HOME

/opt/sonar_scanner

☐ Install automatically ?

Jenkins / v_profile / Configure

Configure

- General
- Source Code Management
- Triggers
- Environment
- Build Steps
- Post-build Actions

⋮ Execute SonarQube Scanner

SonarQube Installation: ?

sonar-server

JDK ?

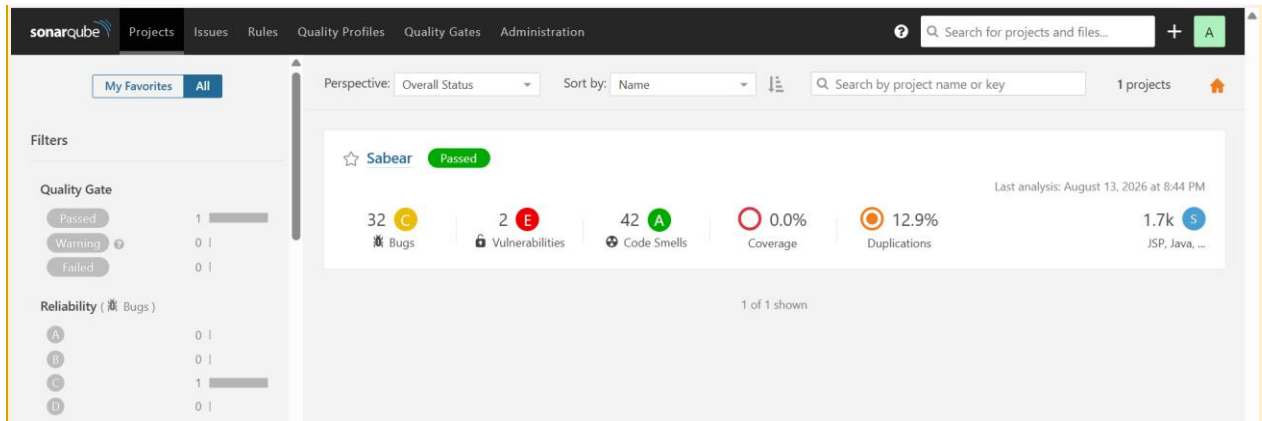
JDK to be used for this SonarQube analysis

(Inherit From Job)

Path to project properties ?

Analysis properties ?

```
sonar-projectversion=1.0
sonar.sources=/var/lib/jenkins/workspace/SJOB_NAME/src/
sonar.binaries=target/classes/com/visualpathit/account/controller/
sonar.junit.reportsPath=target/surefire-reports
sonar.jacoco.reportPath=target/jacoco.exec
sonar.java.binaries=src/main/java/com/visualpathit/account/
```



Step 4: Configure the Slack Integration

Parameter	Value
Slack side	Create a Slack app -> enable Incoming bot token -> invite the bot to the target channel
Jenkins credential	Add the Slack bot token as a Secret text credential, ID slack-token
Location	Manage Jenkins -> System -> Slack
Workspace	khadeergroup.slack.com
Credential	slack-token
Default channel	#jenkins_notification

Click Test Connection and confirm Jenkins reports success before continuing.

Screenshot Evidence: capture the console view for this step and insert it here.

Click Plugin it Open plugin int we have int we have YAML file

Creating your app

1. Go to <https://api.slack.com/apps> and click "Create New App".
2. Click [from an app manifest](#)
3. Select your workspace
4. Delete the example manifest that Slack provides
5. Click YAML
6. Paste the following into the text box:

```
display_information:
  name: Jenkins
features:
  bot_user:
    display_name: Jenkins
    always_online: true
oauth_config:
  scopes:
    bot:
      - channels:read
      - chat:write
      - chat:write.customize
      - files:write
      - reactions:write
      - users:read
```

Labels

[Build Notifiers](#)
[slack](#)

Maintainers

[Joseph Petersen](#)
[Tim Jacomb](#)

Help us improve this page!

To propose a change submit a pull request to the [plugin page](#) on GitHub.

Previous Security Warnings

[CSRF vulnerability and missing permission checks allowed capturing credentials](#)

- Affects version 2.19 and earlier



Create from a manifest



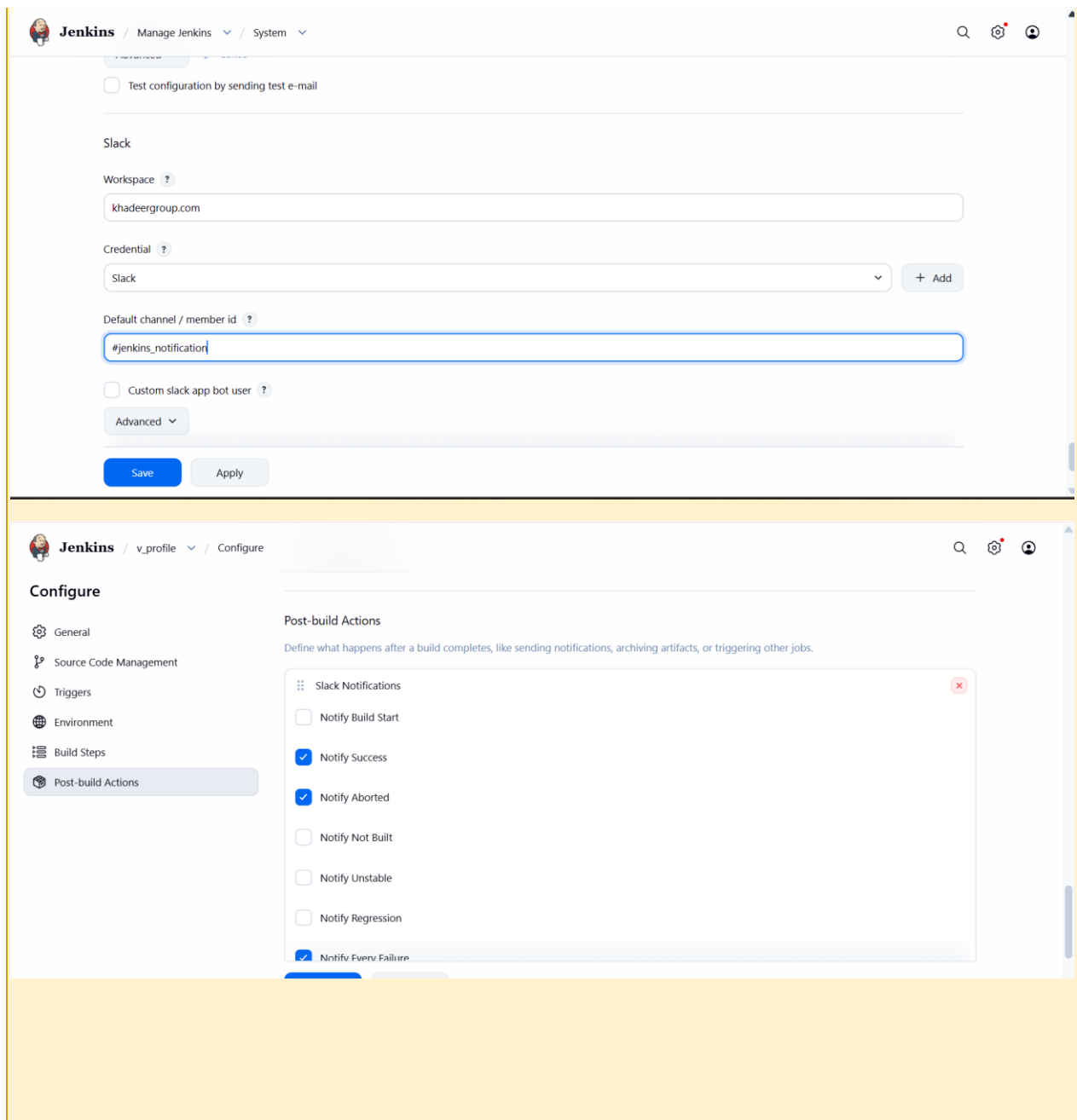
JSON YAML

```
1 display_information:
2   name: Jenkins
3 features:
4   bot_user:
5     display_name: Jenkins
6     always_online: true
7 oauth_config:
8   scopes:
9     bot:
10       - channels:read
11       - chat:write
12       - chat:write.customize
13       - files:write
14       - reactions:write
15       - users:read
16       - users:read.email
17       - groups:read
18 settings:
```

Step 1 of 2

Back

Next



Step 5: Run the Job and Review Results

Click Build Now, then open Console Output to watch all three stages execute in order, and confirm the SonarQube project dashboard and the Slack channel both update.

 **Screenshot Evidence:** capture the console view for this step and insert it here.

The screenshot displays two interfaces. The top interface is the Jenkins console output for a build named 'v_profile' at build number '#5'. The output shows the following log messages:

```
INFO: Jenkins - java -cpD BLOCK_XRUEAET (CORE) / T LINE-2400
INFO: 6 files had no CPD blocks
INFO: Calculating CPD for 14 files
INFO: CPD calculation finished
INFO: Analysis report generated in 59ms, dir size=241 KB
INFO: Analysis report compressed in 30ms, zip size=89 KB
INFO: Analysis report uploaded in 19ms
INFO: ANALYSIS SUCCESSFUL, you can browse http://13.220.59.156:9000/dashboard?id=Sabear
INFO: Note that you will be able to access the updated dashboard once the server has processed the submitted analysis report
INFO: More about the report processing at http://13.220.59.156:9000/api/ce/task?id=AZ\_8KOARFzA-wTghCJHK
INFO: Analysis total time: 7.329 s
INFO: -----
INFO: EXECUTION SUCCESS
INFO: -----
INFO: Total time: 8.140s
INFO: Final Memory: 15M/54M
INFO: -----
[Slack Notifications] found #4 as previous completed, non-aborted build
[Slack Notifications] will send OnSuccessNotification because build matches and user preferences allow it
Finished: SUCCESS
```

The bottom interface is a Slack channel named '#jenkins_notification'. The channel's sidebar shows a list of channels, with '#jenkins_notification' selected. The main chat area shows a message from the Jenkins app at 10:55 PM: 'Slack/Jenkins plugin: you're all set on <http://100.31.57.152:8080/> v_profile - #5 Success after 8.4 sec (Open)'. The message is visible to the user, and the channel's name is '#jenkins_notification'.

Validation Steps

- Console Output shows the Git Clone stage checking out VProfile-1 into the workspace with no authentication errors.
- The SonarQube dashboard shows a new/updated project named VProfile-1 with a completed analysis and a Quality Gate status (Passed or Failed).
- The configured Slack channel receives a message with the build number, job name, and result immediately after the pipeline finishes.
- The Stage View in Jenkins shows all three stages (Git Clone, Sonarqube Integration, Slack Integration) completed in green.

Explanation

The pipeline separates concerns into three linear stages: source retrieval, static code analysis, and notification. Because SonarQube is installed the traditional way (unzipped and run directly with its own embedded Elasticsearch and web server) rather than as a Docker container, Jenkins talks to it over a plain HTTP URL registered once under Manage Jenkins -> System, and the withSonarQubeEnv step injects

that server's URL and token into the scanner call automatically. The Slack stage runs independently of analysis results so the team is notified of both passing and failing builds.

Real-Time Use Case

Teams that are not yet running containerized infrastructure, or that want a single long-lived SonarQube instance shared across many jobs, commonly install SonarQube the traditional way on a dedicated VM and point every Jenkins pipeline at that one server URL, while Slack alerts replace manually checking the Jenkins dashboard after every push.

Common Errors and Troubleshooting

Error	Cause	Resolution
SonarQube fails to start / exits immediately	vm.max_map_count not raised, or Java version unsupported	Apply the sysctl settings in Step 1 and confirm JDK 17 is installed and active
ERROR: Not authorized (SonarQube analysis)	Token missing, expired, or wrong credential selected in Jenkins	Re-generate the token in SonarQube and update the credential used by withSonarQubeEnv
Slack step fails with 'channel_not_found' or no message posted	Bot not invited to the channel, or wrong workspace/credential in Manage Jenkins -> System -> Slack	Invite the Slack app/bot to the channel and re-run Test Connection under Slack settings

Task 2: Pipeline Job for hiring-app (Git Clone, Dockerized SonarQube, Slack Alerts)

Outcome

A second Jenkins pipeline job named hiring-app-sonar-slack runs the same three stages against the hiring-app repository: Git Clone downloads the source code, SonarQube Analysis submits it to a SonarQube server running as a Docker container for a quality scan, and Slack Notification posts the build result to a Slack channel.

Objective

To stand up SonarQube as a Docker container (instead of the Task 1 traditional install) and wire a second, independent application repository into it, confirming the same three-stage integration pattern works against either a traditional or a containerized SonarQube server.

Prerequisites

- Jenkins master with the Pipeline, Git, SonarQube Scanner, and Slack Notification plugins installed (already satisfied from Task 1).
- Docker Engine installed on the host that will run SonarQube, and the jenkins user added to the docker group if Jenkins itself triggers the container.
- At least 2 GB RAM free for the SonarQube container plus its bundled Elasticsearch process.
- The Slack integration already configured under Manage Jenkins -> System -> Slack, with the slack-token credential available.
- Public read access (or credentials) to github.com/betawins/hiring-app.git.

Step-by-Step Implementation

Step 1: Run SonarQube as a Docker Container

Pull and start the official SonarQube image, publishing the web port and using a named volume so data survives container restarts.

```
Yum install docker

Systemctl start docker

Docker pull sonarqube

docker run -itd -p 9001:9000 sonarqube
change the properties file for this specific port
```

 **Screenshot Evidence:** capture the console view for this step and insert it here.

```
[ec2-user@ip-172-31-24-201 ~]$ sudo yum install docker
Last metadata expiration check: 18:31:24 ago on Thu Aug 13 12:20:08 2026.
Dependencies resolved.
```

Package	Architecture	Version	Repository	Size
Installing:				
docker	x86_64	25.0.16-1.amzn2023.0.3	amazonlinux	47 M
Installing dependencies:				
container-selinux	noarch	4:2.245.0-1.amzn2023	amazonlinux	58 k
containerd	x86_64	2.2.5-1.amzn2023.0.1	amazonlinux	24 M
iptables-libs	x86_64	1.8.8-3.amzn2023.0.2	amazonlinux	401 k
iptables-nft	x86_64	1.8.8-3.amzn2023.0.2	amazonlinux	183 k
libcgroup	x86_64	3.0-1.amzn2023.0.1	amazonlinux	75 k
libnetfilter_conntrack	x86_64	1.0.8-2.amzn2023.0.2	amazonlinux	58 k
libnftnl	x86_64	1.0.1-19.amzn2023.0.2	amazonlinux	30 k
libnftnl	x86_64	1.2.2-2.amzn2023.0.2	amazonlinux	84 k
pigz	x86_64	2.5-1.amzn2023.0.4	amazonlinux	83 k
runc	x86_64	1.3.5-1.amzn2023.0.2	amazonlinux	3.9 M

```
Transaction Summary
Install 11 Packages

Total download size: 76 M
Installed size: 284 M
```

```
[ec2-user@ip-172-31-24-201 ~]$ sudo systemctl start docker
[ec2-user@ip-172-31-24-201 ~]$ sudo dcoker pull sonarqube
sudo: dcoker: command not found
[ec2-user@ip-172-31-24-201 ~]$ sudo docker pull sonarqube
Using default tag: latest
latest: Pulling from library/sonarqube
966c395d29cb: Pull complete
e114ca80d2ed: Pull complete
f6c4211cf8b1: Pull complete
4f4fb700ef54: Pull complete
b73294fa41f7: Pull complete
7d7d329955b7: Pull complete
2a4d1635d2a3: Pull complete
Digest: sha256:9dfc19385a3b7647e7585013d8c94ae2322228c6b217a1c641199f07f86b12ab
Status: Downloaded newer image for sonarqube:latest
docker.io/library/sonarqube:latest
```

```
[ec2-user@ip-172-31-24-201 ~]$ sudo docker images
REPOSITORY TAG IMAGE ID CREATED SIZE
sonarqube latest b82115d34675 8 days ago 1.43GB
[ec2-user@ip-172-31-24-201 ~]$ docker run -itd -p 9001:9000 sonarqube
docker: permission denied while trying to connect to the Docker daemon socket at unix:///var/run/docker.sock: Head "http://%2Fvar%2Frun%2Fdocker.sock/_ping"
: dial unix /var/run/docker.sock: connect: permission denied.
See 'docker run --help'.
[ec2-user@ip-172-31-24-201 ~]$ sudo docker run -itd -p 9001:9000 sonarqube
8534292ca815d12b682d84704ebd346132a34863ab1cfa64b7dd66c5eb9c6a7e
[ec2-user@ip-172-31-24-201 ~]$ sudo docker ps
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
8534292ca815 sonarqube "/opt/sonarqube/dock..." 9 seconds ago Up 9 seconds 0.0.0.0:9001->9000/tcp, :::9001->9000/tcp charming_vivsesvaraya
[ec2-user@ip-172-31-24-201 ~]$
```

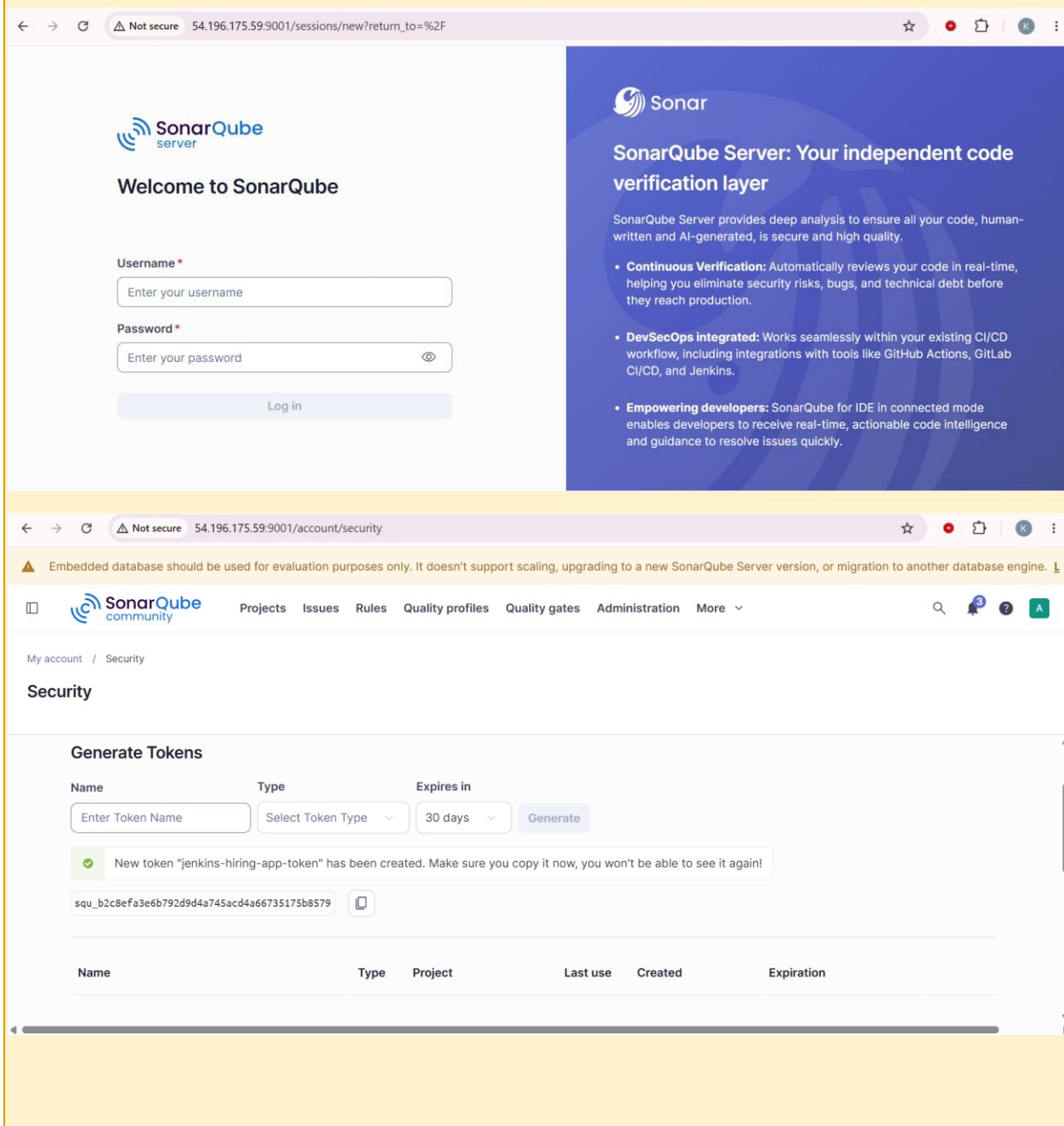
i-0f295c69a58736dfb (sonarQube)

PublicIPs: [54.196.175.59](#) PrivateIPs: 172.31.24.201

Step 2: Verify the Container and Generate a Token

Parameter	Value
URL	http://<docker-host-ip>:9000
Default login	admin / admin (change on first login)
Token location	My Account -> Security -> Generate Tokens
Token name	jenkins-hiring-app-token

 **Screenshot Evidence:** capture the console view for this step and insert it here.



The screenshot shows two browser windows. The top window displays the SonarQube Server login page with fields for Username and Password, and a Log in button. The bottom window shows the SonarQube community interface, specifically the 'Generate Tokens' section under 'My account / Security'. It features a form to create a new token with fields for Name, Type, and Expires in, followed by a 'Generate' button. A success message indicates that a new token 'jenkins-hiring-app-token' has been created. Below the message, the token value 'squ_b2c8efa3e6b792d9d4a745acd4a66735175b8579' is displayed with a copy icon. A table at the bottom lists generated tokens with columns for Name, Type, Project, Last use, Created, and Expiration.

Step 3: Register the Dockerized Server Inside Jenkins

Parameter	Value
Location	Manage Jenkins -> System -> SonarQube servers

Name	sonar-server-docker
Server URL	http://<docker-host-ip>:9000
Server authentication token	Credential built from the token generated in Step 2

This is added as a second entry alongside sonar-server from Task 1; Jenkins can hold multiple SonarQube server definitions and each pipeline picks the one it needs by name.

 **Screenshot Evidence:** capture the console view for this step and insert it here.



List of SonarQube installations

Name

sonar-server

Server URL

Default is http://localhost:9000

http://54.196.175.59:9000/

Server authentication token

SonarQube authentication token. Mandatory when anonymous access is disabled.

SonarQube token for Jenkins

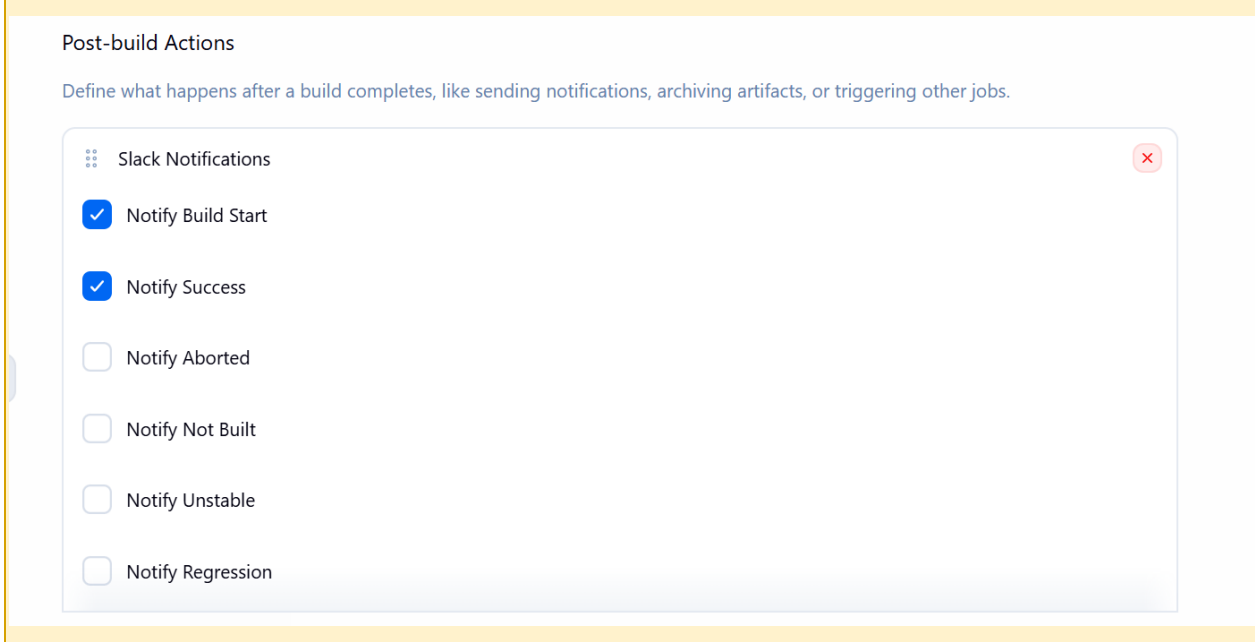
+ Add

Advanced

Step 4: Confirm the Shared Slack Configuration

Open Manage Jenkins -> System -> Slack and confirm the workspace and slack-token credential from Task 1 are still present, since Task 2 reuses the Slack integration rather than creating it again.

 **Screenshot Evidence:** capture the console view for this step and insert it here.



Post-build Actions

Define what happens after a build completes, like sending notifications, archiving artifacts, or triggering other jobs.

Slack Notifications

☒ Notify Build Start

☒ Notify Success

☐ Notify Aborted

☐ Notify Not Built

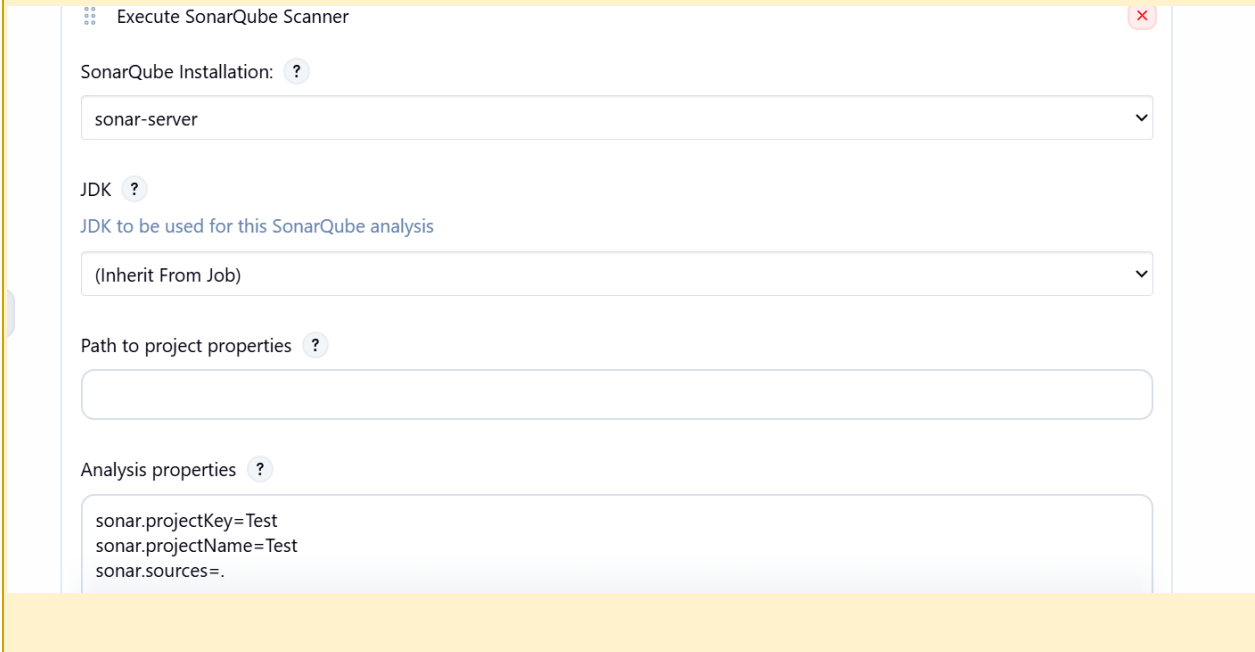
☐ Notify Unstable

☐ Notify Regression

Step 5: Create the Pipeline Job

Parameter	Value
Location	New Item -> free style job
Item name	hiring-app-sonar-slack

 **Screenshot Evidence:** capture the console view for this step and insert it here.

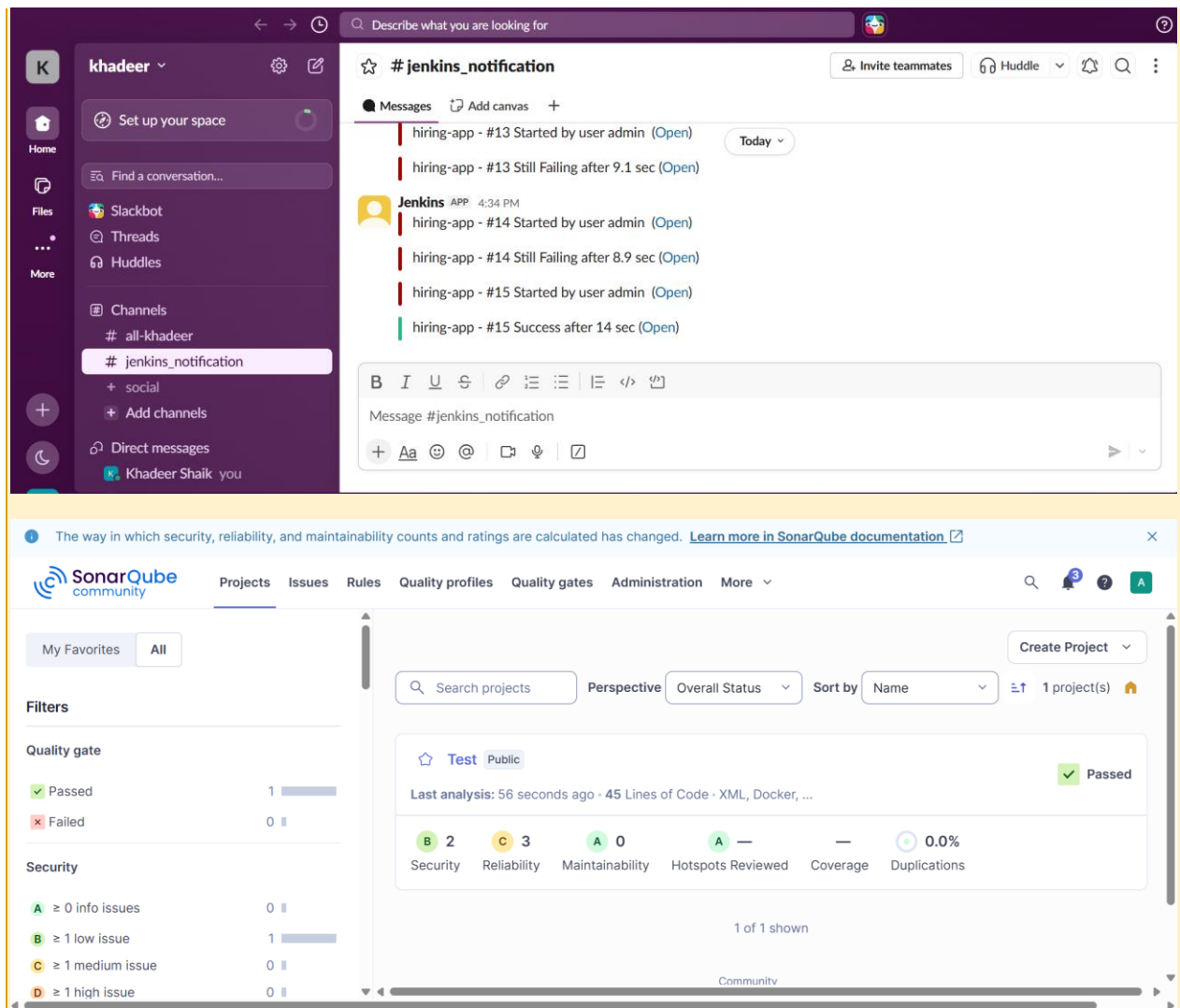


Step 6: Run the Job and Review Results

Click Build Now, then open Console Output to confirm all three stages run in order, and check both the Dockerized SonarQube dashboard and the Slack channel for the new hiring-app project and its build alert.

 **Screenshot Evidence:** capture the console view for this step and insert it here.

```
11:07:13.680 INFO Analysis successful; you can find the results at: http://54.196.175.59:9000/dashboard?id=Test
11:07:13.680 INFO Note that you will be able to access the updated dashboard once the server has processed the submitted analysis report
11:07:13.682 INFO More about the report processing at http://54.196.175.59:9000/api/ce/task?id=48ee712c-762c-4b61-8161-45116d4d115b
11:07:13.701 INFO Analysis total time: 4.414 s
11:07:13.705 INFO SonarScanner Engine completed successfully
11:07:13.729 INFO EXECUTION SUCCESS
11:07:13.730 INFO Total time: 13.028s
[Slack Notifications] found #14 as previous completed, non-aborted build
[Slack Notifications] will send OnSuccessNotification because build matches and user preferences allow it
Finished: SUCCESS
```



Validation Steps

- Console Output shows the Git Clone stage checking out hiring-app into the workspace with no authentication errors.
- The SonarQube dashboard shows a new project named hiring-app with a completed analysis and Quality Gate status, alongside the existing VProfile-1 project.
- The configured Slack channel receives a message for the hiring-app build, distinct from the VProfile-1 build messages.
- The Stage View shows all three stages (Git Clone, Sonarqube Integration, Slack Integration) completed in green.

Explanation

Running SonarQube via docker run instead of the unzip-and-start method in Task 1 gives an isolated, disposable server: the image bundles the correct Java runtime and Elasticsearch configuration, and the named volumes persist data across container restarts or upgrades. Jenkins treats it exactly the same as the traditional install from the pipeline's perspective, it is just another entry (sonar-server-docker) under

Manage Jenkins -> System -> SonarQube servers with its own URL and token, referenced by withSonarQubeEnv. The Slack integration is untouched and reused as-is from Task 1.

Real-Time Use Case

Teams standardizing on containers commonly run SonarQube in Docker (or Docker Compose/Kubernetes) so the server can be recreated, scaled, or moved between hosts with a single command, while keeping Jenkins itself and the traditional-install SonarQube from Task 1 untouched, letting the two coexist as separate server entries for different projects or environments.

Common Errors and Troubleshooting

Error	Cause	Resolution
Container exits immediately / restarts in a loop	Host vm.max_map_count too low for the bundled Elasticsearch	Run <code>sudo sysctl -w vm.max_map_count=262144</code> on the Docker host and restart the container
Jenkins cannot reach <code>http://<docker-host-ip>:9000</code>	Port 9000 not published or blocked by a firewall/security group	Confirm <code>-p 9000:9000</code> was used and the port is open from the Jenkins host
Analysis overwrites the VProfile-1 project in SonarQube	<code>sonar.projectKey</code> left as the default or copied from Task 1	Set a unique <code>sonar.projectKey</code> (hiring-app) in the scanner command
No Slack message for hiring-app builds	Different channel intended but not updated in the Jenkinsfile	Update the channel value in the <code>slackSend</code> steps for this job if a separate channel is required